

CHAPTER 10

10 - NestJS 기초 (모듈 · 컨트롤러 · 서비스 · DTO — 인메모리 CRUD)

Node.js · Express · NestJS 강의 · 2026

NestJS 파트의 출발점. Express(04~09장)에서 손으로 하던 일 — 라우터 분리, 본문 검증, 의존성 연결 — 을 NestJS 는 구조로 강제한다. 이 장은 그 뼈대(모듈·컨트롤러·서비스·DI·DTO)를 게시글(posts) CRUD 하나로 익힌다. 저장은 일부 러 메모리 배열이다 — DB 없이 구조에만 집중한다.

이 장의 `my-app` 은 뼈대 연습용이다. NestJS 파트의 본 프로젝트 ****쇼핑몰(shop)****은 11장에서 새로 시작해 17 장까지 키운다.

이 장에서 배우는 것

- **nest CLI** — `nest new` 로 프로젝트, `nest g resource` 로 CRUD 골격 생성
- **컨트롤러/서비스** — 요청 처리(컨트롤러)와 로직(서비스)의 분리, 데코레이터 라우팅
- **DI(의존성 주입)** — 컨트롤러가 서비스를 `new` 하지 않는다
- **DTO 검증** — `class-validator` + 전역 `ValidationPipe`
- **PartialType** — UpdateDto의 반복 제거

0. 프로젝트 생성 — nest new

```
npm i -g @nestjs/cli           # 전역 설치 (또는 npx @nestjs/cli ... 로 대체)

nest new my-app --skip-git --package-manager npm
cd my-app
npm run start:dev                # → http://localhost:3000 에서 Hello World!
```

이 과정의 코드는 `nest new` 가 만들어 주는 **NestJS 11** 기준이다.

워밍업 — 기본 생성된 `app.controller.ts` / `app.service.ts` 에 라우트를 몇 개 추가해 보자. Express의 `app.get("/say", handler)` 가 NestJS에서는 데코레이터 한 줄이 된다.

```
@Get("/say")                    // app.get("/say", (req, res) => {...})
getSayHi(): string {
  return this.appService.sayHi();
}
```

연습: `GET /profile` → "안녕하세요 개인정보 보여드립니다", `GET /path` → "안녕하세요 경로 안내입니다."

1. posts 리소스 생성 — nest g resource

```
nest g resource posts --no-spec # 물어보면 "REST API" 선택, CRUD entry points 생성 Y
```

한 줄로 `posts/` 아래에 모듈·컨트롤러·서비스·DTO 골격이 생기고, `app.module.ts` 에 자동 등록까지 된다. 생성된 골격의 TODO 부분을 이제 채운다.

2. 코드 작성 순서

순 서	파일	내용
1	<code>src/posts/post.interface.ts</code>	Post 타입 — <code>id · title · content · author</code>
2	<code>src/posts/posts.service.ts</code>	<code>private posts: Post[] = [...]</code> 배열 + <code>nextId</code> 로 CRUD. 없는 id면 <code>NotFoundException (404)</code>
3	<code>src/posts/posts.controller.ts</code>	<code>@Get · @Post · @Patch · @Delete</code> + <code>@Param / @Body</code>
4	<code>src/posts/dto/create-post.dto.ts</code>	<code>@IsString · @MinLength(1)</code> , <code>author</code> 는 <code>@IsOptional</code>
5	<code>src/posts/dto/update-post.dto.ts</code>	<code>extends PartialType(CreatePostDto)</code> — 모든 필드가 선택이 된다
6	<code>src/main.ts</code>	전역 <code>ValidationPipe({ whitelist: true, transform: true })</code>

3. Express ↔ NestJS 대응표

Express (04~09장에서 하던 것)	NestJS (이 장)
<code>app.get("/posts", handler)</code>	<code>@Get()</code> 붙은 메서드
<code>req.params.id</code>	<code>@Param("id") id: string</code>
<code>req.body</code>	<code>@Body() dto: CreatePostDto</code>
<code>res.status(404).json({...})</code>	<code>throw new NotFoundException(...)</code>
미들웨어에서 <code>if (!title) ...</code> 직접 검증	DTO + <code>ValidationPipe</code> 가 자동으로 400
라우터 파일을 손으로 분리·등록	<code>@Module</code> 이 구조를 강제, <code>nest g</code> 가 등록까지

4. 실행 + 동작 테스트 (curl)

```
npm run start:dev # → http://localhost:3000
```

```
BASE=http://localhost:3000
```

```
# 1) 목록 - 초기 데이터 1건
```

```
curl -s $BASE/posts
```

```
# 2) 작성 - author를 빼면 "익명"
```

```
curl -s -X POST $BASE/posts -H "Content-Type: application/json" \
  -d '{"title":"두번째 글","content":"NestJS 배우는 중"}'
```

```
# 3) 검증 실패 - title이 빈 문자열이면 400 (@MinLength(1))
```

```
curl -i -X POST $BASE/posts -H "Content-Type: application/json" -d '{"title":""}' #
400
```

```
# 4) 단건 / 수정 / 삭제
```

```
curl -s $BASE/posts/2
```

```
curl -s -X PATCH $BASE/posts/2 -H "Content-Type: application/json" -d '{"title":"수정됨"}'
```

```
curl -s -X DELETE $BASE/posts/2
```

```
# 5) 없는 글 → 404 (NotFoundException)
```

```
curl -i $BASE/posts/99 #
404
```

핵심 개념

DI — 컨트롤러는 서비스를 만들지 않는다

```
@Controller("posts")
export class PostsController {
  constructor(private readonly postsService: PostsService) {} // 주입받는다
}
```

`new PostsService()` 라고 쓰는 곳이 없다. `@Injectable()` 이 붙은 클래스를 모듈의 `providers` 에 두면, **Nest가** 인스턴스를 만들어 생성자로 넣어 준다. 테스트할 때 가짜 서비스로 바꾸기 쉽고, 서비스가 서비스를 주입받는 협력(11장에 서 바로 쓴다)도 같은 방식이다.

ValidationPipe — 요청 본문의 문지기

`main.ts` 에 전역으로 한 번 걸면 모든 `@Body()` 가 DTO 규칙으로 검증된다.

- `whitelist: true` — DTO에 없는 속성은 버린다 (`{"title":"x","admin":true}` 를 보내도 `admin` 은 사라진다)
- `transform: true` — 들어온 값을 DTO 타입으로 변환

PartialType — UpdateDto를 한 줄로

```
export class UpdatePostDto extends PartialType(CreatePostDto) {}
```

생성 규칙을 물려받되 모든 필드를 선택으로 바꾼다. `@IsOptional` 을 일일이 다시 적지 않는다.

메모리 배열의 한계 → 다음 장의 동기

글을 몇 개 만들고 서버를 재시작해 보라(`start:dev` 면 파일 저장만 해도 재시작된다). **전부 사라진다.** 데이터가 프로세스 메모리에만 있기 때문이다. 11장에서 진짜 데이터베이스(Prisma + SQLite → PostgreSQL)에 저장한다.

핵심 파일

파일	역할
<code>src/posts/posts.service.ts</code>	배열 CRUD + 404 처리 — 로직은 전부 여기
<code>src/posts/posts.controller.ts</code>	라우팅과 파라미터 추출만
<code>src/posts/dto/*.ts</code>	본문 검증 규칙 (<code>class-validator</code>)
<code>src/posts/posts.module.ts</code>	컨트롤러·서비스 묶음
<code>src/app.module.ts</code>	루트 모듈 — <code>PostsModule</code> 조립
<code>src/main.ts</code>	부팅 + 전역 <code>ValidationPipe</code>

따라잡기 — 완성본으로 실행 (보조 경로)

```
cd 10
npm install
npm run start:dev
```

다음 장

- **11 - Prisma 기초:** 쇼핑몰 프로젝트(`shop`)를 시작한다. 상품·분류 CRUD를 Prisma + SQLite로 진짜 저장하고, 마지막에 provider 한 줄을 바꿔 PostgreSQL로 옮긴다.